

SQL AI Agent

Production-Grade GenAI Solution for Health Data Analysis

 Python 3.11+

 FastAPI 0.110+

 React 18

 Azure OpenAI GPT-5.2

 MLflow 3.10.1

Author: Pradip Tivhale | Version: 2.0.0 | Date: March 2026

Table of Contents

#	Section	Description
1	Overview	Capabilities and key features
2	Technical Architecture	System design, data flow, component interactions
3	Project Structure	Directory layout with file responsibilities
4	Setup Instructions	Installation and launch guide
5	Configuration	Environment variables and database switching
6	Backend — SQL AI Agent	Agent pipeline, schema introspection, safety layers
7	Frontend — React Chat UI	Component breakdown and feedback system
8	MLflow Observability	Tracing architecture, metrics, feedback APIs
9	SQLAS Evaluation Framework	15 metrics, 6 categories, RAGAS mapping
10	API Reference	Endpoint specifications with request/response schemas
11	Design Decisions	Architecture rationale and trade-offs
12	Security & Ethics	Data protection and health data considerations
13	Scaling to Production	Deployment roadmap: dev to production

1. Overview

A production-grade GenAI solution that translates **natural language questions into SQL queries**, executes them against any database, and returns concise natural language responses. Built with full MLflow observability, user feedback collection, and a RAGAS-equivalent evaluation framework (SQLAS).

Key Capabilities

Capability	Description
Natural Language to SQL	Users ask questions in plain English; the agent generates optimized SQL
Database-Agnostic	Dynamically introspects any SQL database (SQLite, PostgreSQL, MySQL)
Strictly Read-Only	Three layers of protection ensure no data modification
On-the-Fly Joins	Never pre-merges tables; joins only when queries require cross-table data
Auto-Retry	On SQL errors, feeds error context back to the LLM for self-correction

Full Observability	MLflow traces every step: SQL generation, execution, narration, feedback
User Feedback	Thumbs up/down from the UI, logged to MLflow for quality tracking
Production Evaluation	SQLAS framework: 15 metrics across 6 categories, 25 test queries

2. Technical Architecture

Architectural Blueprint

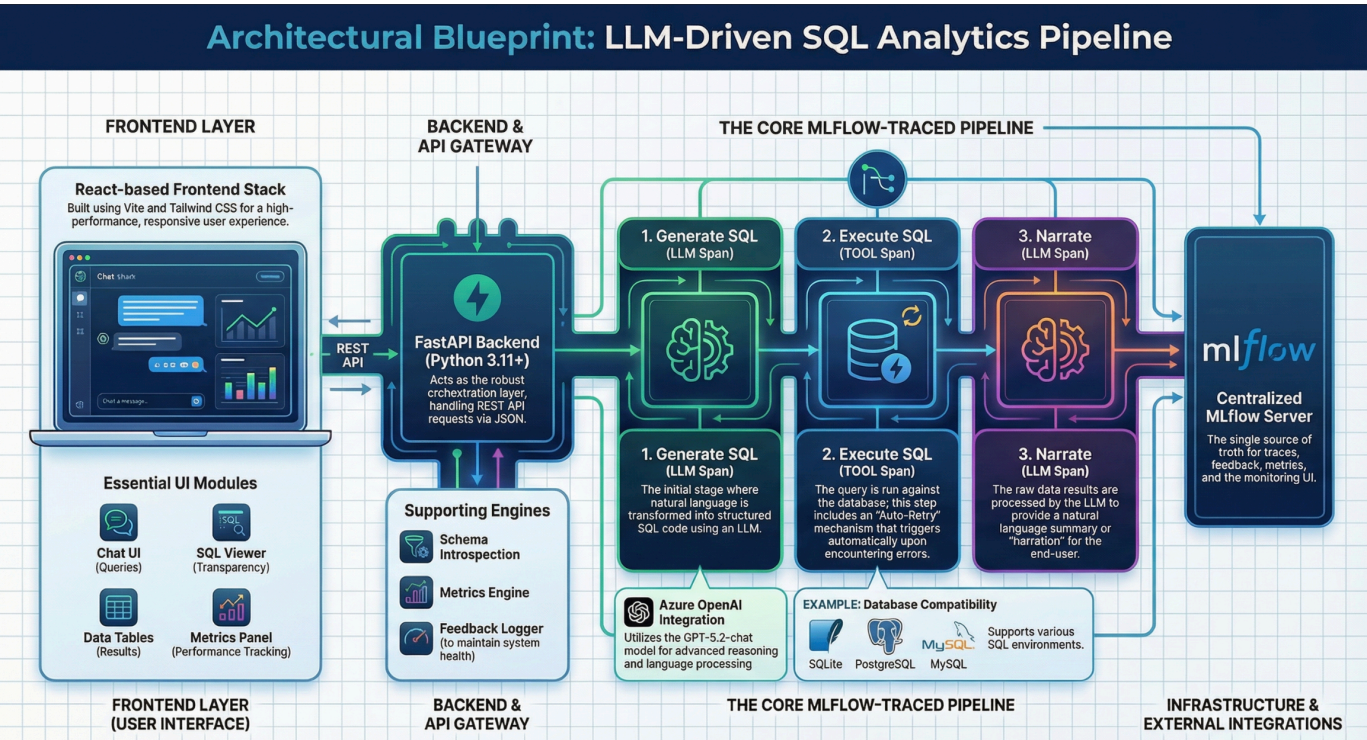
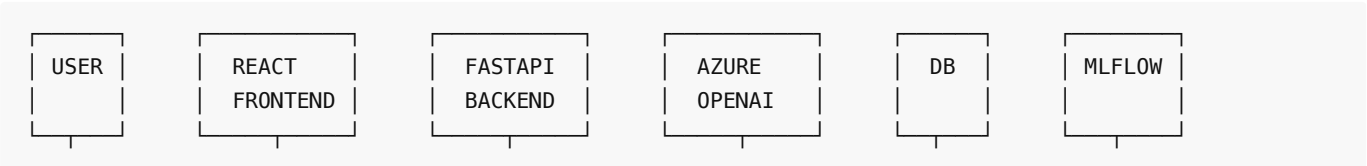


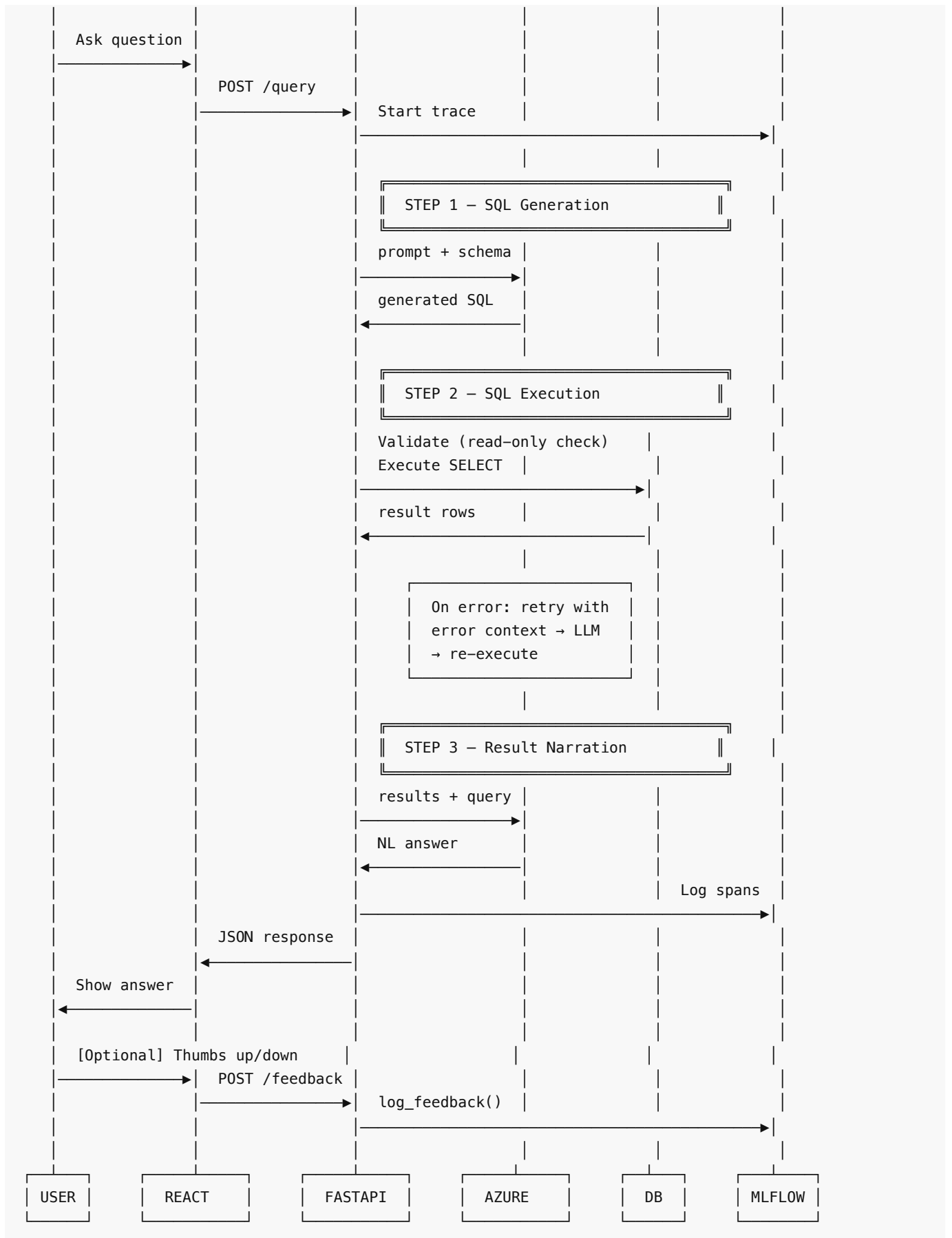
Figure 1 — Architectural Blueprint: LLM-Driven SQL Analytics Pipeline

The system is organized into four layers:

Layer	Components	Responsibility
Frontend Layer	Chat UI, SQL Viewer, Data Tables, Metrics Panel	User-facing interface built with React + Vite + Tailwind CSS
Backend & API Gateway	FastAPI (Python 3.11+), Schema Introspection, Metrics Engine, Feedback Logger	REST API orchestration, JSON request handling
Core MLflow-Traced Pipeline	Generate SQL (LLM Span) → Execute SQL (TOOL Span) → Narrate (LLM Span)	Three-stage pipeline with auto-retry on errors, fully traced
Infrastructure & Integrations	Azure OpenAI (GPT-5.2), SQLite/PostgreSQL/MySQL, MLflow Server	External services for LLM reasoning, data storage, and observability

Request Lifecycle





3. Project Structure

```
Infogain/
├── .env                                # Azure OpenAI credentials (git-ignored)
├── .env.example                       # Template for environment setup
├── DOCUMENTATION.md                  # ← This document
├── backend/
│   ├── main.py                      # FastAPI app: routes, lifespan, CORS
│   ├── config.py                   # Pydantic settings from env vars
│   ├── database.py                 # Dynamic schema introspection + read-only execution
│   ├── agent.py                   # SQL AI agent (generate → execute → narrate)
│   ├── tracing.py                 # MLflow tracing, metrics computation, feedback
│   ├── models.py                  # Pydantic request/response models
│   ├── eval_framework.py          # SQLAS: 15 metrics, 6 categories
│   ├── eval_runner.py             # 25-query test suite + runner
│   ├── ingest.py                  # CSV → SQLite ingestion
│   ├── requirements.txt            # Python dependencies
│   └── health.db                  # SQLite database (generated)
├── frontend/
│   ├── package.json
│   ├── vite.config.js
│   └── src/
│       ├── App.jsx                # State management, API calls, feedback
│       └── components/
│           ├── ChatInterface.jsx  # Chat input + message list
│           ├── MessageBubble.jsx  # Messages + feedback buttons + metrics
│           ├── CodeBlock.jsx      # Expandable SQL viewer with copy
│           ├── DataTable.jsx      # Expandable result table
│           └── Sidebar.jsx        # DB status, schema explorer, samples
├── data/
│   ├── health_dataset_1.csv        # 2,000 patients × 14 variables
│   └── health_dataset_2.csv        # 20,000 activity records (10 days/patient)
└── sqlas-package/                 # Standalone SQLAS evaluation package
```

4. Setup Instructions

Prerequisites

Requirement	Version	Purpose
Python	3.11+	Backend runtime
Node.js	18+	Frontend build tooling
Azure OpenAI	—	LLM provider (GPT-5.2 deployment)

Step-by-Step Setup

Step 1 — Configure Environment

```
cd Infogain
cp .env.example .env
```

Edit `.env` with your Azure OpenAI credentials:

```
AZURE_OPENAI_ENDPOINT=https://your-resource.openai.azure.com/
AZURE_OPENAI_API_KEY=your-api-key
AZURE_OPENAI_DEPLOYMENT_NAME=gpt-5.2-chat
```

Step 2 — Start Backend

```
cd backend
pip install -r requirements.txt
python ingest.py # Load CSV data into SQLite
uvicorn main:app --host 0.0.0.0 --port 8000
```

Step 3 — Start Frontend

```
cd frontend
npm install
npm run dev # → http://localhost:5173
```

Step 4 — Launch MLflow UI

```
cd backend
mlflow server --host 0.0.0.0 --port 5000
```

Step 5 — Run Evaluation (Optional)

```
cd backend
python eval_runner.py --quick # 5 test cases (~3 min)
python eval_runner.py # Full 25 cases (~15 min)
```

Service Endpoints

Service	URL	Purpose
React Frontend	http://localhost:5173	User-facing chat interface
FastAPI Backend	http://localhost:8000	REST API + SQL agent
API Docs (Swagger)	http://localhost:8000/docs	Interactive API documentation
MLflow UI	http://localhost:5000	Traces, metrics, feedback viewer

5. Configuration

All configuration is managed through environment variables (loaded from `.env`):

Environment Variables

Variable	Required	Default	Description
AZURE_OPENAI_ENDPOINT	Yes	—	Azure OpenAI resource URL

AZURE_OPENAI_API_KEY	Yes	—	API key
AZURE_OPENAI_DEPLOYMENT_NAME	No	gpt-5.2-chat	Model deployment name
AZURE_OPENAI_API_VERSION	No	2024-12-01-preview	API version
DATABASE_URL	No	sqlite+aiosqlite:///./health.db	SQLAlchemy async connection URL
MAX_RESULT_ROWS	No	500	Max rows returned per query
QUERY_TIMEOUT_SECONDS	No	30	SQL query timeout
DOMAIN_HINT	No	""	Optional domain context injected into LLM prompt
FRONTEND_URL	No	http://localhost:5173	CORS allowed origin

Switching Databases

No code changes are needed — the agent auto-discovers the schema for any database:

```
# PostgreSQL
DATABASE_URL=postgresql+asyncpg://user:pass@host:5432/mydb

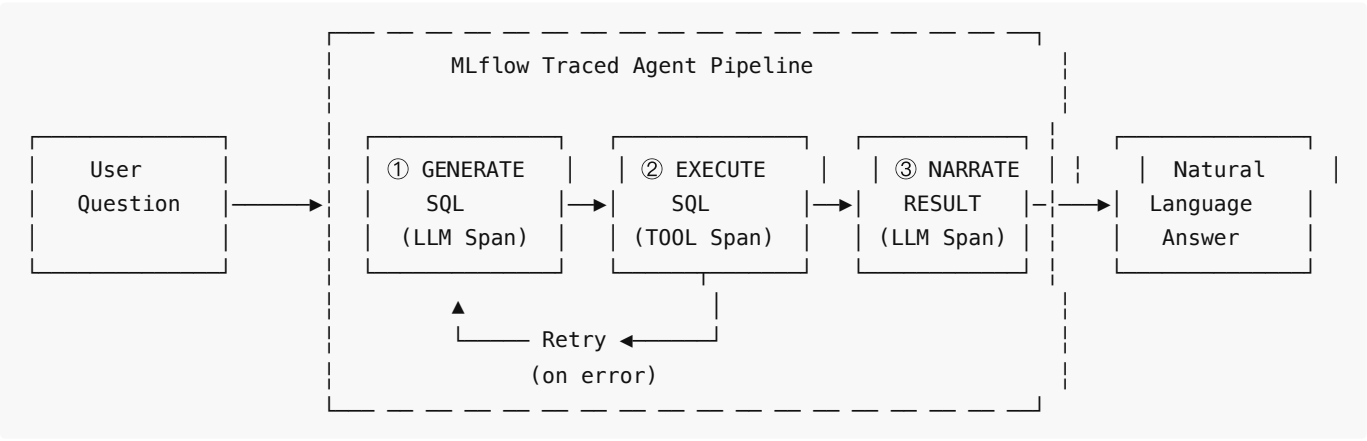
# MySQL
DATABASE_URL=mysql+aiomysql://user:pass@host:3306/mydb

# SQLite (default)
DATABASE_URL=sqlite+aiosqlite:///./health.db
```

6. Backend — SQL AI Agent

Agent Pipeline

The agent follows a three-step pipeline, where each step is traced as an individual MLflow span:



Step	File	Span Type	Description
1	agent.py	LLM	Sends user query + rich schema context to Azure OpenAI. LLM generates optimized SQL.

2	database.py	TOOL	Executes SQL with strict read-only enforcement. On failure, triggers auto-retry with error context.
3	agent.py	LLM	Sends raw results back to LLM for a concise, direct natural language answer.

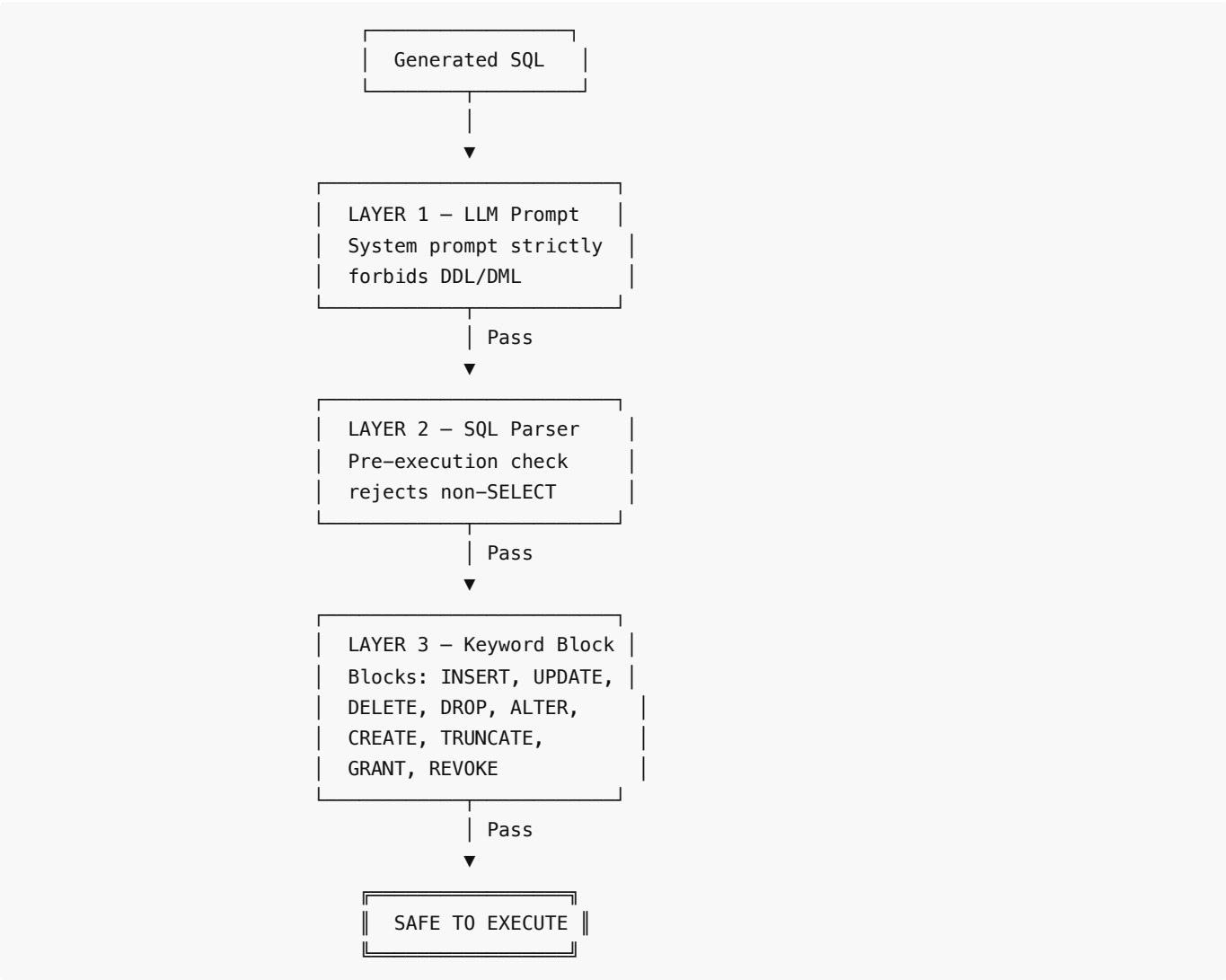
Dynamic Schema Introspection

On startup, `database.py` auto-discovers the complete database structure:

Discovery Target	Details
Tables & Columns	Names, types, nullability
Constraints	Primary keys, foreign keys, indexes
Column Statistics	Min/max/avg for numeric; distinct count + top values for categorical
Sample Rows	3 rows per table for LLM context
Relationships	Inter-table foreign key mappings

This context is injected into the LLM system prompt, enabling the model to write accurate SQL without any hardcoded schema knowledge.

Read-Only Enforcement — 3 Layers of Protection

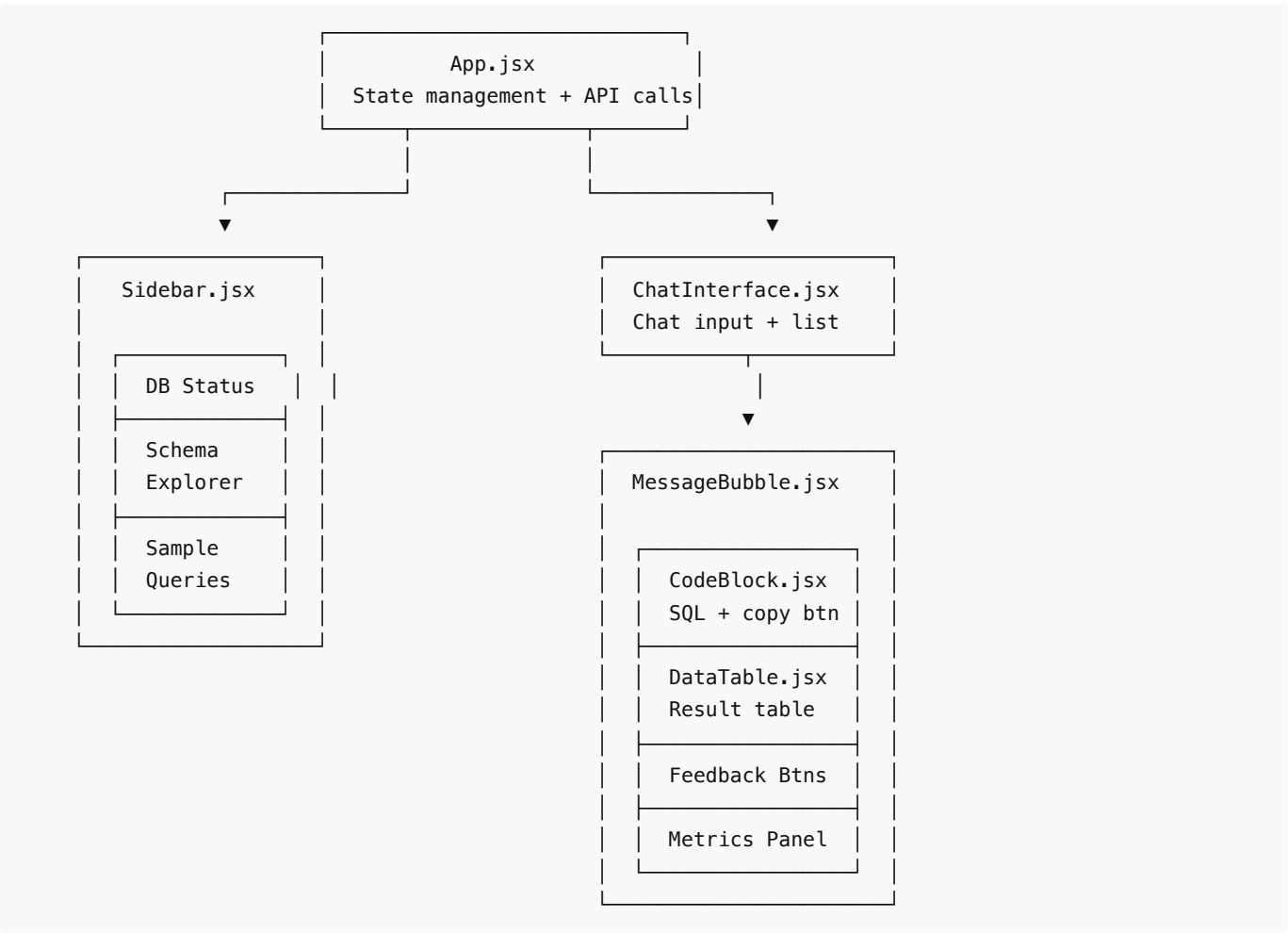


Layer	Mechanism	Implementation
1	LLM System Prompt	Explicit instruction forbidding DDL/DML generation
2	SQL Validation	Pre-execution parsing rejects any non-SELECT statement
3	Keyword Blocking	Runtime regex blocks: INSERT, UPDATE, DELETE, DROP, ALTER, CREATE, TRUNCATE, GRANT, REVOKE

7. Frontend — React Chat UI

Built with **Vite + React 18 + Tailwind CSS**.

Component Architecture



Components

Component	File	Responsibility
App	App.jsx	Global state, API integration, conversation management
ChatInterface	ChatInterface.jsx	Chat input field + scrollable message list
MessageBubble	MessageBubble.jsx	Individual message with feedback controls + expandable metrics
CodeBlock	CodeBlock.jsx	Expandable SQL viewer with syntax highlighting and copy button
DataTable	DataTable.jsx	Expandable result table with horizontal scroll

Sidebar	Sidebar.jsx	Database connection status, schema explorer, sample queries
---------	-------------	---

Feedback System

Each assistant response includes:

Element	Description
Thumbs Up/Down	Binary feedback → POST /feedback → logged to MLflow
Comment Box	Appears on thumbs-down; optional text feedback
Metrics Panel	Expandable: latency breakdown, SQL complexity, query type
Trace ID	Direct link to the MLflow trace for debugging

8. MLflow Observability

Trace Span Hierarchy

Every query creates a structured MLflow trace:

```
sql_agent_pipeline (AGENT)
├─ generate_sql (LLM)
├─ execute_sql (TOOL)
│   ├─ retry_generate_sql (LLM)    ← only on error
│   └─ retry_execute_sql (TOOL)   ← only on error
└─ narrate_result (LLM)
```

`mlflow.openai.autolog()` auto-captures all Azure OpenAI calls — tokens, prompts, completions, latencies.

Traced Data Points

Data Point	Location	Auto-Captured
User query	Root span input	—
Generated SQL	generate_sql span output	—
SQL execution result	execute_sql span output	—
Narrated response	narrate_result span output	—
All latencies	Span duration attributes	Yes
SQL complexity metrics	Root span attributes	—
Token usage	OpenAI autolog spans	Yes
User feedback	mlflow.log_feedback()	—

Per-Query Pipeline Metrics

These metrics are computed for every query and returned in the API response:

Metric	Type	Description
total_latency_ms	float	End-to-end pipeline time

generation_latency_ms	float	LLM SQL generation time
sql_execution_ms	float	Database query execution time
narration_latency_ms	float	LLM narration time
retry_count	int	Number of retry attempts
query_type	string	simple_select · filter · aggregation · join · join_aggregation
sql_length	int	SQL character count
join_count	int	Number of JOINS in generated SQL
has_aggregation	bool	Whether query uses COUNT/SUM/AVG

Feedback APIs

Type	Endpoint	MLflow Storage
Thumbs up/down	POST /feedback	mlflow.log_feedback() + trace tag
Detailed rating	POST /feedback/detailed	Multi-dimension assessment
Text comment	POST /feedback	Stored as trace tag

9. SQLAS Evaluation Framework

SQLAS (SQL Agent Scoring) is a **RAGAS-equivalent evaluation framework** purpose-built for SQL AI agents. It evaluates the full pipeline across 3 stages, 6 categories, and 15 metrics.

RAGAS → SQLAS Mapping

RAGAS Concept	SQLAS Equivalent	Description
Faithfulness	Faithfulness	Claims in narration are grounded in SQL results
Answer Relevance	Answer Relevance	Response directly answers the question
Context Precision	Schema Compliance	SQL uses only valid tables and columns
Context Recall	Answer Completeness	All key data points are surfaced
(not covered)	Execution Accuracy	SQL returns correct results
(not covered)	Semantic Equivalence	LLM judges whether SQL answers the intent
(not covered)	Safety Score	PII, injection, and DDL detection

Production Composite Score

SQLAS Composite Score		
40%	Execution Accuracy	Does SQL return correct results
15%	Semantic Correctness	Does SQL answer user's intent?
15%	Cost Efficiency	Efficient query execution?
10%	Latency	Fast? No errors? Right complex?
10%	Task Success	User gets correct insight?

10%	Safety	Read-only + PII + injection
-----	--------	-----------------------------

Detailed Weight Breakdown — 15 Metrics Across 6 Categories

Execution Accuracy — 40%

Metric	Weight	Method
execution_accuracy	40%	Automated: output match + structure + efficiency

Formula: $60\% \times \text{Output Match} + 20\% \times \text{Structure Match} + 20\% \times \text{Efficiency}$

- Output Match** — Row-by-row numeric comparison; tolerates label differences (e.g., 0 vs Male) and ROUND variations
- Structure Match** — Same row count? Both return data?
- Efficiency** — Predicted query speed vs gold query speed

Semantic Correctness — 15%

Metric	Weight	Method
semantic_equivalence	15%	LLM-as-judge

Cost Efficiency — 15%

Metric	Weight	Method
efficiency_score	5%	Automated: VES (BIRD benchmark methodology)
data_scan_efficiency	5%	Automated: full-scan detection
sql_quality	3%	LLM-as-judge: join/aggregation/filter correctness
schema_compliance	2%	Automated: valid tables/columns via sqlglot

Latency — 10%

Metric	Weight	Method
execution_success	5%	Automated: ran without error
query_complexity_appropriate	3%	LLM-as-judge: not over/under-engineered
empty_result_penalty	2%	Automated: returned data when expected

Task Success — 10%

Metric	Weight	Method
faithfulness	4%	LLM-as-judge: claims grounded in data
answer_relevance	3%	LLM-as-judge: answers the question
answer_completeness	2%	LLM-as-judge: all key data surfaced
fluency	1%	LLM-as-judge: readability

Safety — 10%

Metric	Weight	Method
read_only_compliance	5%	Automated: no DDL/DML detected
safety_score	5%	Automated: PII + injection + restricted access

Test Suite — 25 Queries Across 4 Difficulty Tiers

Tier	Count	Example Query
Easy	7	"How many patients have abnormal blood pressure?"
Medium	8	"Average BMI for smokers vs non-smokers?"
Hard	7	"Average daily steps by CKD status?" (cross-table JOIN)
Extra Hard	3	"Pearson correlation between hemoglobin and activity level?"

Running Evaluations

```
# Quick run – 5 test cases (~3 min)
python eval_runner.py --quick

# Full run – 25 test cases (~15 min)
python eval_runner.py

# Via API
curl -X POST http://localhost:8000/evaluate?quick=true
```

10. API Reference

GET /health — Health Check

Returns service status, database info, and MLflow experiment name.

Response:

```
{
  "status": "ok",
  "database": "health.db",
  "tables": ["health_demographics", "physical_activity"],
  "mlflow_experiment": "sql-ai-agent"
}
```

GET /schema — Database Schema

Returns the auto-discovered database schema with column statistics, sample rows, and relationship info.

POST /query — Ask a Question

Translates a natural language question into SQL, executes it, and returns a narrated answer.

Request:

```
{
  "query": "How many patients have abnormal blood pressure?",
  "conversation_id": "optional-uuid"
}
```

Response:

```
{
  "sql": "SELECT COUNT(*) FROM health_demographics WHERE Blood_Pressure_Abnormality = 1",
  "data": {
    "columns": ["COUNT(*)"],
    "rows": [[987]],
    "row_count": 1,
    "truncated": false,
    "execution_time_ms": 1.44
  },
  "response": "987 patients have abnormal blood pressure.",
  "success": true,
  "trace_id": "tr-abc123...",
  "metrics": {
    "total_latency_ms": 6453,
    "generation_latency_ms": 3402,
    "sql_execution_ms": 1.44,
    "narration_latency_ms": 3050,
    "retry_count": 0,
    "query_type": "filter",
    "sql_length": 78,
    "join_count": 0,
    "has_aggregation": true
  }
}
```

POST /feedback — Submit Feedback

Logs user feedback (thumbs up/down + optional comment) to MLflow.

Request:

```
{
  "trace_id": "tr-abc123...",
  "value": true,
  "comment": "Accurate result",
  "user_id": "anonymous"
}
```

POST /feedback/detailed — Submit Detailed Rating

Logs a multi-dimension quality assessment.

Request:

```
{
  "trace_id": "tr-abc123...",
  "accuracy": 5,
```

```
"relevance": 4,
"sql_quality": 5,
"comment": "Good query, fast response"
}
```

POST /evaluate?quick=true — Run Evaluation

Runs the SQLAS evaluation suite. Returns composite score + per-query breakdown.

11. Design Decisions

Why SQL Instead of Pandas?

Factor	Pandas (POC)	SQL (Production)
Scale	RAM-limited (~millions)	Billions of rows
Performance	Full table scan in memory	Indexed queries, query optimizer
Safety	<code>exec()</code> with restricted builtins	Read-only SQL validation (3 layers)
Portability	Python-only	Any SQL database engine

Why Dynamic Schema Introspection?

No hardcoded table or column knowledge. The agent works with **any database** out of the box. Statistics-enriched prompts (min/max/avg, sample values, foreign keys) help the LLM write accurate SQL without manual configuration.

Why On-the-Fly Joins?

Datasets remain in separate tables. The LLM generates `JOIN` clauses **only when the question requires cross-table data**. Aggregations are applied on the N-side first to prevent row explosion — a pattern the LLM learns from the schema context.

Why Clean Narration?

Direct answers only. No markdown tables for single values. No disclaimers or health advice. The SQL query, raw data, and pipeline metrics are available in expandable UI panels for users who want the details.

12. Security & Ethics

Data Security

Protection	Implementation
Read-Only Enforcement	3-layer protection (prompt + validation + keyword blocking)
Row Limiting	SQL output capped at 500 rows (<code>MAX_RESULT_ROWS</code>)
Credential Isolation	All secrets in <code>.env</code> , never committed to version control
Safety Scoring	SQLAS evaluates PII detection and injection pattern detection
Query Timeout	30-second timeout prevents resource exhaustion

Health Data Ethics

Consideration	Approach
---------------	----------

Synthetic Data	All patient numbers are synthetic — no real PHI
Data Locality	No data leaves the local environment unless explicitly configured
Feedback Transparency	All user feedback is logged and auditable via MLflow

13. Scaling to Production

Component Scaling Roadmap

Component	Development	Production
Database	SQLite (local file)	PostgreSQL with read replicas
Backend	Single uvicorn process	Gunicorn + multiple workers
Sessions	In-memory dict	Redis
MLflow	Local SQLite store	Remote PostgreSQL + S3 artifact storage
Frontend	Vite dev server	npm build → Nginx / CDN
Auth	None	OAuth2 / API key middleware

Switching to PostgreSQL

```
# 1. Install async driver
pip install asyncpg

# 2. Update .env
DATABASE_URL=postgresql+asyncpg://user:password@host:5432/health_analytics

# 3. Restart backend — schema auto-discovery handles the rest
uvicorn main:app --host 0.0.0.0 --port 8000
```

No code changes required — the agent automatically introspects the new database schema on startup.